

전자전기컴퓨터설계실험II

실험 전 보고서

LAB4

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 10월 09일
대상	LEDD와 버튼
검증 환경	VS Code / NUCLEO-G474RE

1. 실험 목적과 관련 이론

실험 목적

NUCLEO-G474RE의 사용자 버튼을 읽고 LED의 상태 변경 간격을 프로그램으로 조절한다. 과제 1에서는 PA5 내장 LED를 사용하고, 과제 2에서는 같은 동작을 PB0 외부 LED에 적용한다. 버튼을 누를 때마다 간격이 1초, 2초, 4초, 8초, 1초로 순환하도록 설계한다.

GPIO 입력과 출력

GPIO는 핀을 디지털 입력 또는 출력으로 사용하는 기능이다. 수업 프로젝트에서 PC13은 사용자 버튼, PA5는 내장 LED, PB0는 외부 출력으로 설정되어 있다. PA5는 HIGH일 때 켜지고 LOW일 때 꺼진다.[3][4]

HAL_GPIO_ReadPin()으로 버튼 상태를 읽고 HAL_GPIO_TogglePin()으로 LED 출력을 반전한다. GPIO_PIN_SET과 GPIO_PIN_RESET은 각각 HIGH와 LOW를 뜻한다.

폴링과 시간 기준

메인 루프에서 polling_routine()을 반복 호출한다. HAL_GetTick()의 밀리초 값을 읽어 마지막 출력 변경 시각과 비교한다. HAL_Delay()로 실행을 멈추지 않으므로 점멸 간격을 기다리는 동안에도 버튼 입력을 확인할 수 있다.

임베디드 시스템 교재는 LED 점멸, 버튼 입력, 디바운싱을 입출력, 타이머 설계의 연결된 주제로 다룬다.[1] 본 실험도 입력 검출, 시간 판단, 출력 변경으로 기능을 나누어 구성한다.

간격과 전체 주기

상태 변경 간격을 Δt라고 하면 켜짐과 꺼짐이 각각 Δt 동안 유지된다. 따라서 전체 점멸 주기는 $T = 2\Delta t$이고 주파수는 $f = 1/(2\Delta t)$이다. 과제의 1초 간격은 전체 주기 2초를 의미한다.

버튼의 누른 순간 검출

현재 버튼 값이 HIGH이고 이전 값이 LOW일 때만 새 입력으로 처리한다. 이전 값을 static 변수에 보관하므로 버튼을 누르고 있는 동안에는 간격이 반복해서 증가하지 않는다.

첫 입력은 즉시 허용하고, 이후 마지막 유효 입력에서 200ms가 지나지 않은 반복 검출은 무시한다. 이 처리는 수업 예제의 간단한 접점 바운스 억제 방식이다. 200ms 이상 안정된 입력을 보장하는 필터와는 다르므로 실제 버튼에서 중복 입력 여부를 확인한다.[4]

실험 장치와 개발 환경

항목	사용 내용
보드	NUCLEO-G474RE
입력	PC13 사용자 버튼 B1
과제 1 출력	PA5 내장 LED
과제 2 출력	PB0 외부 LED
외부 부품	LED, 330 Ω~1 kΩ 저항, 브레드보드, 점퍼선
연결	ST-LINK USB 데이터 케이블
소프트웨어	VS Code, STM32 HAL, CMake, Ninja, ARM GCC·GDB

주요 도구 버전

- 컴파일러: Arm GNU Toolchain 14.3.Rel1
- 빌드 도구: CMake 4.4.0과 Ninja 1.13.2

- ST-LINK GDB Server: 7.14.0
- STM32CubeProgrammer: 2.23.0

2. 과제 1 내장 LED 간격 변경

설계 목표와 동작 흐름

버튼이 없어도 선택한 간격으로 계속 점멸하고, 한 번 누를 때 한 단계만 변경한다. 시작 간격은 1000ms이며 상태 순환은 1000 → 2000 → 4000 → 8000 → 1000ms이다.

현재 시각, 버튼 읽기 → 새 누름 검출 → 간격 갱신 → 경과 시간 확인 → LED 반전 순서로 동작한다. 버튼 처리와 LED 시간 판단은 같은 함수 호출에서 순서대로 수행한다.

상태 변수와 입력 읽기

```
static uint32_t interval_ms =
1000U;
static uint32_t last_toggle_ms =
0U;
static uint32_t last_press_ms =
0U;
static uint8_t press_seen = 0U;
static GPIO_PinState previous =
    GPIO_PIN_RESET;

uint32_t now_ms = HAL_GetTick();
GPIO_PinState current =
HAL_GPIO_ReadPin(
    BUTTON_ece2_GPIO_Port,
    BUTTON_ece2_Pin);
```

함수 안의 static 변수는 다음 호출에서도 값을 유지한다. interval_ms는 선택 간격, last_toggle_ms는 마지막 LED 변경 시각, last_press_ms는 마지막 유효 버튼 시각이다.

버튼 입력에 따른 간격 변경

```
if (current == GPIO_PIN_SET
    && previous == GPIO_PIN_RESET
    && (press_seen == 0U
        || now_ms-last_press_ms >=
200U))
{
    last_press_ms = now_ms;
    press_seen = 1U;
    interval_ms = (interval_ms ==
8000U)
        ? 1000U : interval_ms *
2U;
    last_toggle_ms = now_ms;
}
previous = current;
```

위의 코드는 polling_routine()의 입력 처리 부분이다.

선택한 간격으로 계속 점멸

```
if    (now_ms-last_toggle_ms    >=
interval_ms)
{
    last_toggle_ms = now_ms;
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
LED_ece2_Pin);
}
```

과제1에서의출력 처리 부분으로 버튼을 누를 때 last_toggle_ms를 현재 시각으로 갱신한다. 따라서 누른 직후 LED가 즉시 반전되는 것이 아니라, 현재 상태를 유지한 채 새로 선택한 간격이 지난 뒤 반전된다.

파라미터 계산과 예상 결과

선택 간격	전체 주기	주파수
1 s	2 s	0.5 Hz
2 s	4 s	0.25 Hz
4 s	8 s	0.125 Hz
8 s	16 s	0.0625 Hz

검증할 경계 조건

버튼을 계속 누르는 경우, 누른 뒤 놓는 경우, 8초에서 1초로 돌아오는 경우를 구분해 확인한다. 놓는 순간에는 간격이 바뀌지 않아야 한다. 200ms이내 재검출은 무시되므로 정상 반복 시험은 충분히 간격을 두고 수행한다.

시간 차이는 uint32_t의 뺄셈으로 계산한다. 짧은 주기로 폴링한다는 조건에서 카운터가 최댓값을 넘어 순환해도 경과 시간을 비교할 수 있다. 실제 전환에는 루프 처리 시간에 따른 작은 지연이 포함될 수 있다.

입력 조건별 검증 계획

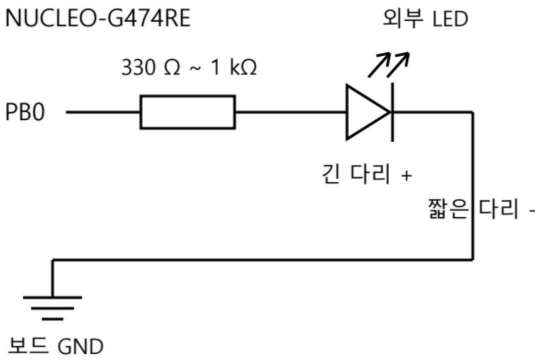
입력·관찰 조건	기대 결과
전원 인가 또는 리셋	간격 1초로 시작
버튼 없이 계속 관찰	현재 간격으로 반복 점멸
버튼을 한 번씩 4회 누름	2 → 4 → 8 → 1초
버튼을 길게 누름	누른 순간 한 단계만 변경
버튼을 놓음	간격 유지
200ms 이내 추가 검출	추가 간격 변경 억제
8초 상태에서 새 입력	1초 상태로 순환

3. 과제 2 외부 LED 동작

설계 목표

과제 1의 버튼 처리와 시간 제어를 유지하고 출력 대상만 PB0에 연결한 외부 LED로 변경한다. 이를 통해 프로그램의 제어 논리와 실제 GPIO 핀 연결의 관계를 확인한다.

외부 LED 회로 구성



위의 그림[5]은 PB0 외부 LED의 직렬 연결을 나타낸 것으로 LED의 극성과 보드 GND 연결을 확인한다.

USB 전원을 분리한 상태에서 PB0 → 저항 → LED 애노드(긴 다리) 순서로 연결한다. 캐소드(짧은 다리)는 보드 GND에 연결한다.

직렬 저항의 역할과 계산

직렬 저항은 LED에 흐르는 전류를 제한한다. 저항에 걸리는 전압에 옴의 법칙을 적용한 전류의 근삿값은 I = (VOUT - VF)/R이다. LED 자체는 비선형 소자이므로 저항처럼 일정한 비례 관계를 가정하지 않는다.[2]

VOUT = 3.3V, LED 순방향 전압 VF = 2.0V를 예로 들면 330Ω에서는 약 3.9mA, 1kΩ에서는 약 1.3mA이다. 이는 부품 선정 이해를 위한 예시이며 실제 VF와 출력 전압은 사용 LED와 구동 조건에 따라 달라진다.

핀 이름과 코드 변경

PB0는 GPIO_OUT1_ece2_GPIO_Port와 GPIO_OUT1_ece2_Pin으로 정의되어 있다. 과제 1의 내장 LED 포트·핀 대신 이 이름을 사용한다. PB1을 선택한다면 OUT1을 OUT2로 바꾼다.[4]

다음 코드는 같은 소스에서 빌드 옵션에 따라 출력 핀 선택함을 보여준다.

```
#if
defined(STM32_0_EXTERNAL_LED_PB0)
    HAL_GPIO_TogglePin(
        GPIO_OUT1_ece2_GPIO_Port,
        GPIO_OUT1_ece2_Pin);
#else
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
        LED_ece2_Pin);
#endif
```

과제별 빌드 설정

설정	대상	외부 LED 옵션
Debug	PA5	OFF
ExternalLED	PB0	ON

CMakeLists.txt에 STM32_0_EXTERNAL_LED_PB0 옵션을 추가하고, CMakePresets.json에서 과제별 ON/OFF 값을 지정하였다. 두 설정 모두 디버깅 정보를 포함한다. 출력 파일은 각각 build/Debug와 build/ExternalLED 폴더에 생성된다.

```
option(STM32_0_EXTERNAL_LED_PB0
"Use PB0 external LED for STM32_0
task 2" OFF)

if(STM32_0_EXTERNAL_LED_PB0)
   target_compile_definitions
(${CMAKE_PROJECT_NAME}
PRIVATE STM32_0_EXTERNAL_LED_PB0)
endif()
```

이 파일을 추가하여 이 옵션을 켜면 외부 LED 코드가 빌드되고 끄면 내장 LED 코드가 빌드되어서 C파일을 덮어쓰지 않아도 되도록 설계하였다.

보드 확인 순서

① 내장 LED 과제의 확인을 마치고 Shift+F5로 디버깅을 종료한다.
② USB 전원을 분리하고 외부 LED 회로를 연결한다.
③ USB를 다시 연결한 뒤 STM32 과제 2: 외부 LED (PB0)를 선택한다.
④ F5로 빌드와 다운로드한다. main에서 멈추면 F5로 계속 실행한다.
⑤ 외부 LED의 점멸과 버튼에 따른 간격 순환을 확인한다.

예상 결과

외부 LED에서도 1 → 2 → 4 → 8 → 1초 순서로 상태 변경 간격이 바뀌어야 한다. 내장 LED와 외부 LED를 동시에 점멸시키는 것이 아니라, 선택한 과제의 LED를 제어한다.

4. 문제 해결 과정과 결론

문제	수정 내용
CPU 종류 인식 오류	디버그 대상 arm 명시
실행 파일 미지정	다운로드 전에 ELF와 심볼 로딩
실행 파일 경로 오류	GDB 명령 경로를 C:/ 형식으로 지정
빌드 작업을 찾지 못함	Configure·Build 작업을 workspace에 통합

위 수정은 개발 환경과 다운로드 절차의 문제를 해결한 것이다. GPIO의 핀 연결이나 실제 점멸 시간 검증을 대신하지 않는다.

5. 소스 구성과 GitHub 주소

주요 파일의 역할

파일	역할
ece2_474_routine.c	버튼 검출, 간격 갱신, LED 반전
main.c / main.h	기존 초기화, 메인 루프와 핀 정의
CMakeLists.txt	외부 LED 선택 옵션
CMakePresets.json	과제별 빌드 설정
ece2_474.code-workspace	과제 실행 메뉴, 빌드·디버그 연결

실습 코드는 Core/Src/ece2_474_routine.c의 polling_routine()에 작성하였다. main.c, main.h와 HAL 드라이버의 기본 구조는 유지했다.

소스 저장소와 제출 기준

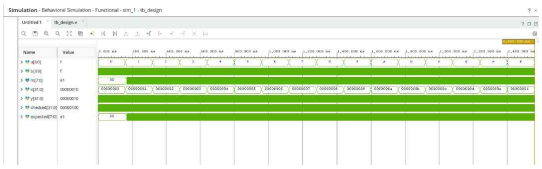
- GitHub 계정: thechef030
- 저장소: STM32_0
- 주소: https://github.com/thechef030/STM32_0
- Branch: stm32-0
- Tag: v1.1.0
- 코드 커밋: 14caa8a
- 전체 커밋 해시: 14caa8a56b244905436d1b7e1efb89844341ffa

6. 실험1 4비트 곱셈기

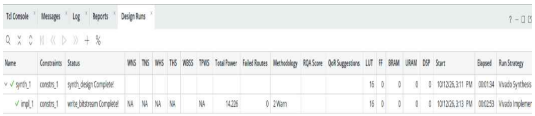
목표

4비트이진수 두 개를 입력받아 곱셈 결과를 8비트로 출력한다.

Vivado 실험결과



비트스트림 생성 결과



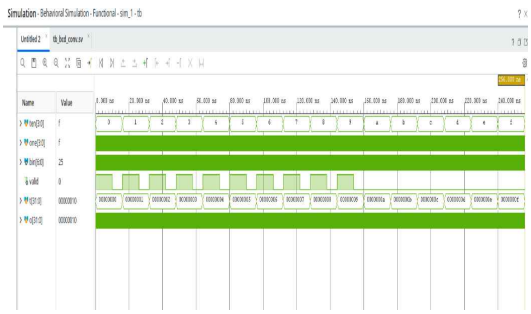
7. 실험2 BCD 이진수 변환

목표

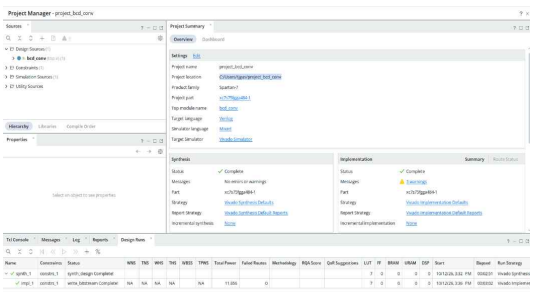
BCD로 표현된 십진수 두 자리를 일반 이진수로 변환한다.

(binary = 10 X ten + one)

Vivado 실험 결과



비트스트림 생성 결과



8. 참고문헌

[1] Elecia White, *Making Embedded Systems*, 2nd ed., O'Reilly Media, 2024. ISBN 9781098151539. Ch. 4 "Inputs, Outputs, and Timers", Ch. 5 "Interrupts".

[2] Samuel J. Ling, William Moebs, Jeff Sanny, *University Physics Volume 2*, OpenStax, 2016. §9.4 "Ohm's Law".

[3] STMicroelectronics, *STM32G4 Nucleo-64 boards (MB1367)*, UM2505, Rev. 7, 2026.

[4] STM32_4week_plan.pdf
(전전설2 LAB4 교안)

[5] chatgpt를 활용한 그림 생성